

MergeConflictBench: Evaluating LLM Agents on Semantically Correct Merge Conflict Resolution

Daniel Chen¹

¹Anything, Inc.

Abstract

We introduce MERGECONFLICTBENCH, an evaluation suite for measuring the ability of LLM agents to resolve real merge conflicts while preserving behavior from both branches. The benchmark consists of 86 cases extracted from production merge conflict tracking in a commercial React-family codebase—React.js and Next.js web surfaces, React Native mobile surfaces, and adjacent JavaScript/JSX API and helper modules. The corpus spans 288 conflicted files with 1,063 conflict blocks across domains including authentication, mobile UI, regulatory data processing, animation systems, fleet management, e-commerce, and CRM tooling. Each case is paired with a hidden holdout test suite—2,540 executable tests in total across all 86 cases—that verifies behaviors originating from all three code lineages: the common base, the “ours” branch, and the “theirs” branch. An agent’s task is to produce resolved files free of conflict markers; its output is scored by executing the hidden tests, directly measuring whether the resolution correctly integrates both branches’ contributions. This design extends the principle established by REFACTORBENCH—*behavioral preservation is a functional property best verified by functional tests*—from single-branch refactoring to multi-branch integration, where the agent must synthesize semantically compatible code from two divergent change sets. Release-time validation executes all 2,540 hidden tests against the checked-in reference resolutions, with all 86 cases passing; deterministic accept-ours, accept-theirs, and accept-both baselines pass 17, 10, and 48 cases respectively. A one-shot LLM block-replacement harness across five contemporary models passes between 25 and 36 cases, showing that semantic merge resolution remains challenging even when models can inspect the full conflicted file contents.

Keywords: benchmark, React.js, merge conflict resolution, large language models, agentic evaluation, behavioral preservation

1 Introduction

Merge conflicts are an unavoidable consequence of collaborative software development. When two branches modify overlapping regions of the same file, version control systems cannot automatically reconcile the changes and instead produce *conflict markers*—the familiar <<<<<<, =====, and >>>>>> delimiters that bracket the competing versions. A developer must then manually inspect each conflict block and produce a resolution that correctly integrates the intent of both branches.

For trivial conflicts—a renamed variable on one side, an added import on the other—resolution is mechanical. But many real-world conflicts require *semantic understanding*: when both branches restructure the same function, add interacting features, or change overlapping state management logic, the correct resolution is not a simple union of the two sides but a careful synthesis that preserves the behavior introduced by each branch while maintaining overall program coherence.

Three common resolution strategies illustrate the difficulty spectrum:

- **Accept ours / accept theirs.** Discard one branch’s changes entirely. Correct only when one side’s changes are strictly superseded.
- **Accept both.** Concatenate both sides. Correct only when the changes are independent and order-insensitive.
- **Custom resolution.** The developer writes new code that integrates both branches’ intent. This is the non-trivial case.

LLM-based agents can potentially resolve conflicts that require semantic understanding, but evaluating whether an agent’s resolution is correct poses the same challenge that motivated REFACTORBENCH [Chen, 2025]: structural comparison (exact match, diff size) is unreliable for code transformations where multiple textually distinct outputs can be functionally equivalent. A resolution that reorders function parameters, uses a different variable name, or restructures a conditional may be perfectly correct despite differing from the reference.

We address this with hidden functional tests. Each benchmark case includes a test suite—invisible to the resolving agent—that exercises behaviors from all three code lineages: the common ancestor (base), the current branch (ours), and the incoming branch (theirs). A resolution passes if and only if it preserves all three categories of behavior.

This paper introduces MERGECONFLICTBENCH, a benchmark of 86 React-family merge conflict cases from production, and makes the following contributions:

1. **MergeConflictBench itself:** a benchmark of 86 real React.js/Next.js/React Native merge conflict cases spanning 288 files and 1,063 conflict blocks, with 2,540 executable hidden tests covering behaviors from base, ours, and theirs branches across all cases.
2. **Extension of behavioral evaluation to merge resolution:** we demonstrate that the hidden-test methodology from REFACTORBENCH transfers naturally to the merge conflict domain, where it must verify the *synthesis* of two change sets rather than the *preservation* of one.
3. **Computed release-time and model baselines:** we report a full reference-resolution validation, three deterministic merge-strategy baselines, and a five-model one-shot LLM matrix, with all counts generated from checked-in benchmark artifacts or the released Escher eval harness.

2 Related Work

Behavioral evaluation of code transformations. REFACTORBENCH [Chen, 2025] introduced hidden holdout test suites for evaluating behavior-preserving code decomposition, demonstrating that functional tests detect regressions invisible to compilation, code similarity, and LLM-as-judge metrics. MERGECONFLICTBENCH extends this methodology from single-branch transformations (refactoring) to multi-branch integration (merge conflict resolution), where the correctness criterion is not “preserve existing behavior” but “synthesize the behaviors of two divergent branches.”

Agentic coding benchmarks. SWE-bench [Jimenez et al., 2024] evaluates agents on resolving real GitHub issues, using repository test suites as ground truth. SWE-agent [Yang et al., 2024] demonstrated that agent tool design significantly affects performance on SWE-bench. OpenHands [Wang et al., 2024] provides an open platform for coding agents. Aider’s code editing

benchmarks [Gauthier, 2024a,b] use test-based evaluation for refactoring and editing tasks across languages. These benchmarks evaluate *generative* coding tasks (write new code to fix an issue); MERGECONFLICTBENCH evaluates an *integrative* task (combine two existing change sets correctly).

Merge conflict resolution. Traditional merge tools (kdiff3, meld, IntelliJ’s merge resolver) provide three-way diff views but leave resolution to the developer. DeepMerge [Dinella et al., 2022] trains sequence-to-sequence models on merge resolution data, evaluating by exact match against reference resolutions. MergeBERT [Svyatkovskiy et al., 2022] uses a token-level classification approach. MergeGen [Zhang et al., 2024] applies retrieval-augmented generation. These approaches evaluate by *textual similarity* to reference resolutions—a metric that can over-penalize correct resolutions that differ from the reference and cannot verify semantic correctness. MERGECONFLICTBENCH is, to our knowledge, the first merge conflict benchmark that evaluates by *executing behavioral tests* against the resolution.

Semantic merge tools. Semantic merge approaches [Apel et al., 2011] use language-aware AST merging to reduce spurious conflicts but do not handle semantic conflicts where both branches modify the same logic. Program synthesis approaches to merge [Sousa et al., 2018] can verify correctness via formal specifications but are limited to restricted program fragments. MERGECONFLICTBENCH targets the general case where arbitrary code changes must be integrated.

3 Benchmark Design

3.1 Design Principles

MERGECONFLICTBENCH is built on four design principles:

1. **Functional evaluation.** The primary metric is whether hidden holdout tests pass after resolution, directly measuring whether the agent correctly integrated both branches’ contributions. Structural metrics (no remaining conflict markers, compilation success) are necessary conditions recorded as secondary scores.
2. **Three-lineage behavioral coverage.** The hidden test suite for each case covers behaviors from all three code lineages—base, ours, and theirs. A resolution that simply accepts one side will fail the tests for the other side’s behaviors. A resolution that accepts both without proper integration may fail tests that exercise interactions between the two branches’ changes.
3. **Real production conflicts.** All cases are extracted from production merge conflict tracking, not synthesized or curated from open-source repositories. This ensures the conflicts represent the kinds of semantic entanglement that occur in real collaborative development.
4. **Non-trivial resolution required.** Cases are selected for conflicts that were resolved via `custom_resolution` or `accepted_both` strategies in production—not `accepted_ours` or `accepted_theirs`, which are trivial.

3.2 Data Source

Cases are extracted from production merge conflict tracking stored in BigQuery. The source tables contain 3.3 million conflict tracking rows (`generation.fs_conflict_results`) and 109,000

resolution tracking rows (`generation.merge_fs_conflict_resolution_results`). The extraction pipeline selects resolved conflicts where the resolution strategy was `custom_resolution` or `accepted_both`, filters for cases with at least two conflict blocks (to exclude trivially small conflicts), and retrieves the conflicted files (with markers) and the reference resolution.

3.3 Three-Branch Structure

Each benchmark case captures the three-way merge structure:

- **Base:** the common ancestor before either branch diverged.
- **Ours:** the current branch’s version (changes since base).
- **Theirs:** the incoming branch’s version (changes since base).
- **Conflicted:** the merge result with conflict markers, containing both `ours` and `theirs` content delimited by `<<<<<<`, `=====`, and `>>>>>>` markers.

The agent receives only the conflicted files. The base, ours, and theirs versions are used during test authoring but are not provided to the agent at evaluation time. The reference resolution is used as a correctness guide during test authoring but is not part of the scoring—only the hidden tests determine correctness.

3.4 Case Layout

Each case directory follows a consistent structure:

```
data/<case_name>/
  eval.config.json           # case metadata
  conflict_eval.config.json  # conflict blocks + test config
  conflicted/                # files with markers (agent input)
  resolved/                  # reference resolution (test authoring)
  merge.test.js              # hidden test suite (holdout)
```

The `eval.config.json` records the case name, description, source conflict ID, timestamp, language, number of conflicted files, and total conflict block count. The `conflict_eval.config.json` enumerates every conflict block (with ours/theirs content), lists the conflicted file paths, specifies the test file, and documents the preserved behaviors as a structured contract.

3.5 Hidden Test Suite Design

Each hidden test suite is authored by a senior engineer following these principles:

- **Test observable behavior from all three lineages.** Test suites are authored against behaviors identified from the base, ours, and theirs lineages, and some behavior-contract entries include explicit origin tags where that attribution was recorded.
- **Cover the preserved-behavior contract.** The `preservedBehaviors` field in each case’s config serves as the contract: a list of human-readable behavior descriptions. The test suite is the executable form of this contract.

- **Use stable assertions.** Test by return values, API responses, rendered output, and state transitions—not by internal variable names or code structure, which may differ in a correct alternative resolution.
- **Exercise interactions.** Where the two branches’ changes interact (e.g., both modify the same data structure, both add validation rules to the same endpoint), include tests that verify the combined behavior, not just each branch’s changes in isolation.

3.6 Scoring

MERGECONFLICTBENCH produces five scores per resolution attempt:

Table 1: Scoring metrics for MERGECONFLICTBENCH.

Score	Type	Description
Passes Tests	Binary	Do the hidden holdout tests pass against the resolved files? <i>Primary metric.</i>
Resolution Reported Success	Binary	Did the resolver produce a complete marker-free candidate?
Block Resolution Coverage	Ratio	Fraction of configured conflict blocks for which the resolver produced a parseable replacement.
No Conflict Markers	Binary	Output contains no remaining <<<<<< / ===== / >>>>>> markers.
Usage Available	Binary	Did the provider route report usage metadata for the model call?

The *Passes Tests* score is the primary metric. The other scores provide diagnostic information: a resolution that passes tests but has remaining conflict markers would indicate a test suite gap; partial block coverage distinguishes malformed outputs from behavior-preservation failures; a resolution that the resolver reports as successful but that fails tests reveals miscalibration.

4 Corpus

The corpus consists of 86 cases totaling 288 conflicted files, 1,063 conflict blocks, and 2,540 executable hidden tests. All cases are JavaScript/JSX drawn from React-family application code: React.js and Next.js web surfaces, React Native mobile surfaces, and adjacent API/helper modules that support those applications. Block counts range from 2 to 46, file counts from 1 to 8, and per-case test counts from 9 to 94. Counts are generated from the checked-in configs and reference-validation report using `scripts/summarize-corpus.mjs`. Table 2 presents a representative sample of 40 cases spanning all complexity tiers, followed by aggregate statistics for the full corpus.

Table 2: MERGECONFLICTBENCH corpus (representative sample). Files = number of conflicted files. Blocks = total conflict blocks. Tests = executable hidden holdout tests per case. Domain describes the functional area of the code.

Case	Tier	Files	Blocks	Tests	Domain
fleet_parts_transfers	Complex	4	46	44	Fleet parts transfer workflow
state_parser_apply	Complex	1	45	40	Regulatory data import API
account_statements	Complex	2	36	35	Account statement generation
fleet_parts_receipts	Complex	3	34	34	Fleet parts receipt processing
stats_home_page	Complex	2	33	29	Statistics dashboard home
mobile_auth_flow	Complex	4	32	55	Mobile auth via WebView
dispatch_sidebar_nav	Complex	2	31	28	Dispatch sidebar navigation
account_auth_pages	Complex	4	30	63	Signup/signin/reset pages
state_parser_preview	Complex	8	25	94	Admin preview with registry
campaign_mapping_tool	Complex	7	25	63	Campaign canvas & toolbar
listing_watchlist	Complex	3	25	22	Listing watchlist feature
celebration_overlay	Complex	1	24	72	Confetti animation & audio
mobile_chat_profile	Complex	5	22	20	Mobile chat profile views
bulk_product_status	Complex	3	22	32	Bulk status change & preview
legal_compliance_settings	Complex	2	22	32	EULA/SMS document URLs
mobile_task_detail	Complex	5	21	62	Task modal with comments
route_mileage_planning	Complex	3	21	34	Mileage estimation hooks
navigation_layout_tailwind	Complex	3	20	31	Navigation layout styling
resource_crud_modals	Moderate	7	19	29	Resource CRUD modal forms
checkout_payment_verify	Moderate	3	19	28	Checkout payment verification
trade_show_form	Moderate	1	19	83	Mobile trade show creation
dealer_integration_csv	Moderate	3	17	27	Dealer CSV integration
onboarding_agreement	Moderate	4	16	60	PDF/HTML agreement flow
invoice_create_modal	Moderate	4	15	46	Invoice creation modal
mobile_tab_navigation	Moderate	4	14	45	Bottom tab bar & badges
ai_studio_animation	Moderate	2	13	40	AI studio animations
org_switcher	Moderate	3	12	60	Org switcher with RBAC
legal_compliance_v2	Moderate	2	12	25	Document link storage
globe_renderer_animation	Moderate	4	10	33	Globe renderer animation
auth_api_routes	Moderate	4	8	52	Login/signup/OTP routes
billing_pricing_pages	Simple	3	7	18	Billing & pricing pages
athlete_contact_api	Simple	1	6	30	Contact endpoint & email rules
signin_academy_service	Simple	2	6	22	Signin & academy service
password_reset_full	Simple	5	5	22	Password reset flow (full)
mobile_customer_filters	Simple	4	4	47	Customer search & team queries
presales_copilot_products	Simple	4	4	39	Presales copilot products
campaign_call_panel	Simple	1	4	31	Call panel & speaker tracking
sme_leader_domains	Simple	3	3	18	SME leader domains
product_import_modal	Simple	1	2	50	Field mapping for imports
matchmaking_user_helpers	Simple	2	2	12	Matchmaking user helpers
<i>Remaining 46 cases</i>		<i>(see supplementary data for full inventory)</i>			
Full corpus (86 cases)		288	1,063	2,540	

4.1 Complexity Distribution

Cases are stratified into three complexity tiers based on the number of conflict blocks:

- **Complex (20+ blocks):** 18 cases (21%), containing 514 conflict blocks across 62 files with 790 hidden tests. These involve extensive interleaving of changes, often requiring the agent to understand how both branches restructured the same functions, state management, or API surfaces. The most complex case (`fleet_parts_transfers`) has 46 conflict blocks across 4 files—a fleet parts transfer workflow where both branches restructured transfer logic, receipt handling, and validation. The single-file maximum is `state_parser_apply` with 45 blocks in one file—a regulatory data import API where both branches restructured function signatures, parameter handling, and handler logic.
- **Moderate (8–19 blocks):** 29 cases (34%), containing 370 conflict blocks across 114 files with 956 hidden tests. These involve meaningful conflicts across multiple files but with more localized changes. For example, `onboarding_agreement` (16 blocks across 4 files) involves conflicts in invitation flow state building, parameter passing, and rendering mode (iframe vs. direct). The median case in this tier has 13 blocks across 3 files.
- **Simple (2–7 blocks):** 39 cases (45%), containing 179 conflict blocks across 112 files with 794 hidden tests. Despite having fewer conflict blocks, these cases still require semantic understanding beyond accept-ours/theirs. For example, `athlete_contact_api` (6 blocks in 1 file) involves conflicting changes to email uniqueness rules, contact type checking, and error response formatting. Several simple cases have high test counts (e.g., `mobile_customer_filters` with 47 tests for 4 blocks), reflecting dense behavioral coverage even in structurally small conflicts.

The overall distribution has a median of 8.5 blocks per case (mean 12.4), with the bulk of cases in the simple-to-moderate range. This reflects the natural distribution of merge conflicts in production: most conflicts are localized, but a meaningful tail of highly complex multi-block conflicts exists and poses the greatest challenge for automated resolution.

4.2 Conflict Type Taxonomy

Across the corpus, we observe recurring categories of semantic conflict:

- **Function refactoring conflicts.** Both branches restructure the same function: one extracts parameters into an options object, the other adds new parameters. The resolution must combine the structural change with the new functionality.
- **State management conflicts.** Both branches modify how component state is managed: one renames state variables, the other adds new state fields. The resolution must apply the renaming to both old and new fields.
- **Import/dependency conflicts.** Both branches add imports from different modules or restructure existing imports. Typically the simplest conflict type, but can interact with other conflicts if the imported symbols are used in conflicting code.
- **UI layout conflicts.** Both branches modify the same component’s JSX structure: one changes the layout hierarchy, the other adds new UI elements. The resolution must place the new elements within the revised hierarchy.
- **API contract conflicts.** Both branches change the same API endpoint: one modifies the request format, the other changes the response format. The resolution must be internally consistent across request and response handling.

4.3 Preserved Behavior Contract

Each case includes a `preservedBehaviors` list documenting what the hidden tests verify. Across the released corpus, these configs contain 1,383 human-readable behavior descriptions. Most entries are description-only; entries may also include an optional `origin` tag (base, ours, or theirs) when attribution has been explicitly recorded. For example, the `athlete_contact_api` case has 19 origin-tagged preserved behaviors: 10 from base (existing validation, auth checks, constraint error handling), 4 from ours (new email conflict detection logic), and 5 from theirs (contact type checking, updated error responses). This structure makes coverage gaps visible during test authoring while the executable test suite remains the scoring authority.

5 Evaluation Methodology

5.1 Agent Task

The agent is given the conflicted files—source files containing `<<<<<<< / ===== / >>>>>>>` conflict markers—and asked to produce resolved files. The agent may use any strategy: reading the conflict markers to understand both sides, reasoning about the semantic intent of each branch, and writing resolved code that integrates both contributions. The agent does *not* have access to the hidden test suite, the reference resolution, or the base/ours/theirs versions as separate files.

5.2 Evaluation Pipeline

After the agent produces its resolution, the evaluation pipeline (`scripts/evaluate.ts`) performs the following steps:

1. Copy the agent’s resolved files into a temporary workspace.
2. Copy the hidden test file (`merge.test.js`) into the workspace.
3. Execute the test suite via Vitest.
4. Record which tests pass and which fail.
5. Check for remaining conflict markers in the resolved files.
6. Report the five scoring dimensions (Table 1).

5.3 Scoring Dimensions

The primary metric is *Passes Tests*: a binary indicator of whether all hidden holdout tests pass. This directly measures functional correctness—whether the resolution preserves behaviors from all three lineages.

Secondary metrics provide diagnostic context:

- *No Conflict Markers* detects resolutions where the agent failed to address some conflict blocks, leaving raw markers in the output.
- *Block Resolution Coverage* measures whether the resolver supplied parseable replacements for the configured conflict blocks.

- *Resolution Reported Success* measures resolver calibration—the divergence between a complete marker-free candidate and actual functional correctness.
- *Usage Available* records whether the provider route returned usage metadata, which helps diagnose infrastructure failures separately from semantic failures.

6 Release-Time Validation, Baselines, and Model Matrix

We compute two release-time checks from the public repository. First, we validate the checked-in reference resolutions by running every hidden test suite against each case’s `resolved/` directory. Second, we evaluate three deterministic merge-editor baselines: accept ours, accept theirs, and accept both. These baselines replace every conflict block mechanically and do not perform semantic integration. We additionally report a one-shot LLM matrix using a tiny block-replacement harness in Escher: each model receives the conflicted files only, emits replacement text for each conflict block, and is scored by the same hidden tests.

All results in this section are generated by the commands documented in `data/eval-results/README.md`. The case and tier summaries are generated by `scripts/summarize-corpus.mjs`; the deterministic baselines are generated by `scripts/evaluate-naive-baselines.mjs`; and the model matrix is generated by the Escher merge-conflict eval harness. The raw Vitest JSON reports and CSV summaries are released under `data/eval-results/`.

Table 3: Release-time validation and deterministic baselines on all 86 cases. “Cases Passed” is the primary metric: a case passes only when its full hidden test suite passes. Assertion counts for failed deterministic baselines are lower bounds because syntax or import failures can abort a suite before every assertion runs.

Resolution	Cases Passed	Case Rate	Assertions Passed	Description
Reference <code>resolved/</code>	86 / 86	100.0%	2,540 / 2,540	Checked-in production resolution used for test authoring
Accept ours	17 / 86	19.8%	2,121 before failures	Use only the current-branch side of every conflict block
Accept theirs	10 / 86	11.6%	2,025 before failures	Use only the incoming-branch side of every conflict block
Accept both	48 / 86	55.8%	2,032 before failures	Concatenate ours followed by theirs for every conflict block

6.1 Key Observations

The released reference resolutions are internally validated. All 86 reference resolutions pass their hidden suites, for 2,540 passing executable tests and zero failures. This does not measure agent performance; it validates that the committed reference resolutions and hidden tests are mutually consistent at release time.

Table 4: One-shot LLM block-replacement results on all 86 cases. “Cases Passed” counts missing or unscored rows as failures, because the benchmark metric is performance across the full fixture set. “Blocks Covered” is mean parseable block-replacement coverage; “Marker-free” is the fraction of rows with no remaining conflict markers.

Model	Cases Passed	Case Rate	Blocks Covered	Marker-free	Usage
Claude Opus 4.6	36 / 86	41.9%	87.0%	75.6%	76 / 86
Gemini 2.5 Pro	35 / 86	40.7%	35.5%	27.9%	86 / 86
Claude Sonnet 4.6	30 / 86	34.9%	84.5%	61.6%	76 / 86
Gemini 2.5 Flash	29 / 86	33.7%	65.3%	38.4%	86 / 86
GPT-4.1	25 / 86	29.1%	84.0%	41.9%	86 / 86

Naive merge-editor strategies are insufficient. Accepting one side performs poorly: accept-ours passes 17 cases (19.8%), while accept-theirs passes 10 cases (11.6%). Accept-both is stronger, passing 48 cases (55.8%), but still fails 38 cases. This supports the benchmark’s selection criterion: many cases require semantic synthesis rather than a blanket policy.

Case-level pass rate is the appropriate baseline metric. Failed baseline runs may abort early because the mechanically resolved file has syntax, import, or runtime errors. For this reason, assertion counts are diagnostic only; a case is counted as correct only when the full hidden suite passes.

One-shot LLM resolution is not yet enough. The strongest model in the one-shot block-replacement harness, Claude Opus 4.6, passes 36 of 86 cases (41.9%), below the deterministic accept-both baseline’s 48 cases. This does not imply that LLM agents are worse than naive merging in general: the harness intentionally provides no tools, no test feedback, and no iterative repair loop. It does show that semantic merge resolution is not solved by a single model call that merely replaces conflict blocks.

Diagnostic scores reveal different failure modes. Claude Opus and Sonnet produce high block-replacement coverage but lose cases to provider-route failures and hidden-test mismatches. Gemini 2.5 Pro passes 35 cases despite low marker-free and block-coverage diagnostics, indicating that some test suites do not exercise every unresolved region and that diagnostic scores should be reported alongside the primary hidden-test metric. The direct GPT-4.1 run has complete usage coverage and no provider-route failures, but lower semantic pass rate.

7 Discussion

7.1 Challenges in Test Authoring

Writing hidden test suites for merge conflict cases poses challenges distinct from refactoring test authoring:

- **Three-lineage coverage.** The test author must understand the behavior of three code versions (base, ours, theirs) and verify that all three lineages’ behaviors are preserved in the resolution. This requires careful reading of diffs to identify what each branch introduced.

- **Interaction testing.** When both branches modify the same data structure or API, the test must verify the *combined* behavior, not just each branch’s changes independently. For example, if ours adds email validation and theirs adds phone validation to the same contact endpoint, a test must verify that both validations are active simultaneously.
- **Stable assertions against divergent implementations.** Because multiple textually distinct resolutions can be correct, tests must assert on externally observable behavior (API responses, rendered output, return values) rather than internal structure. This is particularly challenging for UI component conflicts where the correct resolution might have different component hierarchies.

7.2 Interpreting the Baselines

The deterministic baselines are intentionally weak but informative. Accept-ours and accept-theirs measure how often one side subsumes the other; both rates are low. Accept-both measures how often textual concatenation is sufficient; it passes a slim majority of cases but still fails 44.2% of the corpus. These failures include cases where concatenation creates syntax errors, duplicate definitions, inconsistent API contracts, or behavior that compiles but drops an interaction between the two branches.

The deterministic baseline results establish a floor for non-semantic merge policies and validate that many cases require integration decisions that are sensitive to behavior, not just conflict-marker removal. The one-shot LLM results establish a complementary baseline for tool-free model reasoning: models can often produce parseable replacements, but without iterative execution feedback they still miss branch interactions that the hidden tests exercise.

7.3 Comparison with Refactoring Evaluation

MERGECONFLICTBENCH and REFACTORBENCH share the hidden-test methodology but differ in what constitutes correctness:

Table 5: Comparison of behavioral evaluation in refactoring vs. merge conflict resolution.

Dimension	RefactorBench	MergeConflictBench
Task	Decompose one file into modules	Integrate two divergent change sets
Correctness criterion	Preserve all existing behavior	Preserve base + ours + theirs behavior
Agent input	Single source file	Conflicted files with markers
Structural change	Expected (new files created)	Not expected (same files, markers removed)
Failure mode	Dropped behavior from refactoring	Dropped behavior from one branch

A key difference is that merge conflict resolution does not typically change the file structure—the output should be the same files as the input, but with markers replaced by resolved code. This makes the task more constrained than refactoring (where the agent has creative freedom over module boundaries) but also more demanding in terms of semantic understanding (where the agent must synthesize two competing implementations).

7.4 Future Work

- **Tool-using agent evaluation.** The reported model matrix uses a deliberately small one-shot block-replacement harness. Future runs should evaluate full coding agents with file editing, test

execution on public smoke tests, iterative repair, and provider-specific APIs such as OpenAI’s Responses API for models that are not exposed through chat completions.

- **Language and framework diversity.** The current corpus is entirely JavaScript/JSX from React-family applications. Extending to Python, Go, Rust, and non-React frontend/backend frameworks would test generalization, as merge conflict patterns differ across languages and ecosystems (e.g., import conflicts in Python vs. TypeScript, ownership conflicts in Rust, component-state conflicts in React vs. template-driven frameworks).
- **Automated test generation.** Manually authoring hidden test suites is the primary bottleneck. Exploring automated or semi-automated test generation from the three-way diff—potentially using an LLM to draft tests from the `preservedBehaviors` contract, with human review—could scale the benchmark significantly.
- **Larger corpus via open-source repositories.** Extracting merge conflicts from popular open-source repositories with existing test suites could dramatically expand the corpus. If a repository’s CI tests already cover the behavior introduced by each branch, those tests could serve as the hidden holdout suite with minimal additional authoring.
- **Resolution strategy analysis.** Classifying agent resolutions by strategy (accept-ours, accept-theirs, accept-both, custom synthesis) and correlating with correctness would reveal which conflict types agents handle well and which they struggle with.

8 Conclusion

We introduced MERGECONFLICTBENCH, a benchmark for evaluating LLM agents on merge conflict resolution using hidden behavioral tests. The benchmark extends the principle established by REFACTORBENCH—behavioral preservation is a functional property best verified by functional tests—from single-branch refactoring to multi-branch integration.

The corpus consists of 86 real merge conflict cases from production React-family JavaScript/JSX code, spanning 288 conflicted files with 1,063 conflict blocks and 2,540 executable hidden tests across complexity tiers from 2-block import conflicts to 46-block fleet management workflows. All 86 cases have complete hidden test suites covering behaviors from all three code lineages (base, ours, theirs), ensuring that a correct resolution must genuinely integrate both branches’ contributions rather than favoring one side.

Release-time validation confirms that the checked-in reference resolutions pass all hidden tests. Deterministic merge-editor baselines are substantially weaker: accept-ours, accept-theirs, and accept-both pass 17, 10, and 48 of 86 cases respectively. A one-shot LLM model matrix passes between 25 and 36 cases across five models, with Claude Opus 4.6 strongest in this harness at 36 of 86. These results show that the corpus requires more than conflict-marker removal or one-sided selection; a resolver must make semantic integration decisions, and current one-shot model calls still miss many of them.

Merge conflict resolution is a ubiquitous task in software development and a natural target for LLM automation. MERGECONFLICTBENCH provides the evaluation infrastructure needed to develop and compare merge resolution agents with confidence that measured improvements correspond to genuine gains in semantic correctness.

References

- S. Apel, O. Lessening, and C. Lengauer. Structured merge revisited. In *Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE)*, pages 190–201, 2011.
- D. Chen. RefactorBench: Evaluating LLM agents on behavior-preserving code decomposition. Technical report, Anything, Inc., 2025.
- E. Dinella, T. Mytkowicz, A. Svyatkovskiy, C. Bird, and S. Lahiri. Deepmerge: Learning to merge programs. In *Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE)*, 2022.
- P. Gauthier. Aider refactoring benchmark. <https://aider.chat/docs/leaderboards/refactor.html>, 2024.
- P. Gauthier. Aider polyglot benchmark. <https://aider.chat/2024/12/21/polyglot.html>, 2024.
- C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- M. Sousa, I. Dillig, and S. Lahiri. Verified three-way program merge. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):1–29, 2018.
- A. Svyatkovskiy, T. Mytkowicz, E. Dinella, C. Bird, and S. Lahiri. MergeBERT: Program merge conflict resolution via neural multi-task learning. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022.
- X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zhang, B. Yu, J. Bisk, C. Xiong, T. Yu, G. Neubig, and Y. Li. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- Y. Zhang, H. Yu, Z. Sun, and J. Dong. MergeGen: A retrieval-augmented approach for automatic merge conflict resolution. *arXiv preprint arXiv:2403.07766*, 2024.